



AIを学び、実装する
DeepSquare

AIエンジニア育成講座（E資格講座）

Docker 演習

Docker 演習

1. Dockerのインストール
2. dockerコマンドによるイメージ取得とコンテナ稼働
3. Dockerfileからのイメージの作成およびコンテナの起動・停止を行う
4. 深層学習向けフレームワークが利用できるDocker環境上でモデルの訓練を行う。

Docker 演習

1-1.Windowsへのインストール

公式ドキュメント: Docker for Windows Installation

<https://matsuand.github.io/docs.docker.jp.onthefly/engine/install/>

(1) 上記の公式サイトから Docker Desktop for Windows をダウンロードする。

手順 1 : マニュアル / Docker Desktop / Windows / Docker Desktop for Windows のインストール に移動する。



Docker 演習

1-1.Windowsへのインストール

手順 2 : 下にスクロールし、**Windows への Docker Desktop のインストール** に移動



Docker 演習

1-1.Windowsへのインストール

手順 3 : [Docker Hub](https://docs.docker.com/desktop/install/windows-install/) のリンク先 (https://docs.docker.com/desktop/install/windows-install/) から、[Docker Desktop for Windows](#) をクリックすると、インストーラー (Docker Desktop Installer.exe) のダウンロードが始まる。

The screenshot shows the Docker Docs website with the 'Install Docker Desktop on Windows' page selected. The left sidebar contains a navigation menu with items like 'Overview', 'Install Docker Desktop', 'Install on Mac', 'Install on Windows' (highlighted), 'Install on Linux', and 'System requirements'. The main content area has the title 'Install Docker Desktop on Windows' and a sub-header 'Docker Desktop for Windows'. It includes a section for 'Docker Desktop terms' and a 'System requirements' section with a list of requirements: WSL version 1.1.3.0 or later, Windows 11 64-bit Home or Pro version 21H2 or higher, or Windows 10 64-bit Home or Pro 21H2 (build 19044) or higher, and turning on the WSL 2 feature. The right sidebar shows a 'Contents' section with links to 'System requirements', 'About Windows containers', 'Install Docker Desktop on Windows', 'Install interactively', 'Install from the command line', 'Start Docker Desktop', and 'Where to go next'. There is also a 'Related content' section with links to 'Install Docker Desktop on Mac', 'Install Docker Desktop on Ubuntu', 'Linux post-installation steps for Docker Engine', 'Try Docker Compose', and 'Install Docker Desktop on Linux'.

1-1.Windowsへのインストール

(2) インストーラを起動し、指示に従ってインストールを進める。

Windows への Docker Desktop のインストール

対話的なインストール

1. Docker Desktop Installer.exe をダブルクリックしてインストーラを起動します。

インストーラ (`Docker Desktop Installer.exe`) をまだダウンロードしていない場合は [Docker Hub](#) からダウンロードしてください。通常は `Downloads` フォルダにダウンロードされます。あるいはウェブブラウザの下段に表示される、最新のダウンロードバーから直接起動することもできます。

2. プロンプト画面が表示されたら、バックエンドでの選択状況に従って Configuration ページにおける **Use WSL 2 instead of Hyper-V** オプションのオンオフが設定されていることを確認してください。

利用するシステムが 2 つのオプションの 1 つのみをサポートしている場合は、バックエンドにおいて選択することはできません。

3. インストールウィザードの手順に従ってインストーラを承認して、インストールを完了させます。

4. インストールが正常に行われたら **Close** をクリックして作業を終了します。

5. 管理アカウントが利用しているアカウントと異なる場合、そのアカウントを **docker-users** グループに追加する必要があります。 **administrator** (管理者) として **Computer Management** (コンピューターの管理) を実行し、**Local Users and Groups > Groups > docker-users** を選びます。このグループに対して、右クリックメニューからユーザーを追加します。変更を有効にするため、いったんログアウトしてからログインし直します。

Docker 演習

1-2. OS X へのインストール

公式ドキュメント: Docker Desktop for Mac

<https://matsuand.github.io/docs.docker.jp.onthefly/engine/install/>

(1) 上記の公式サイトから Docker Desktop for Mac をダウンロードする。

マニュアル / Docker Desktop / Mac / Docker Desktop for Mac のインストール に移動する。
環境に従って、Intel チップの Mac か App チップの Mac のどちらかを選択してクリックすると、
Docker.dmg のダウンロードが始まる。



Docker 演習

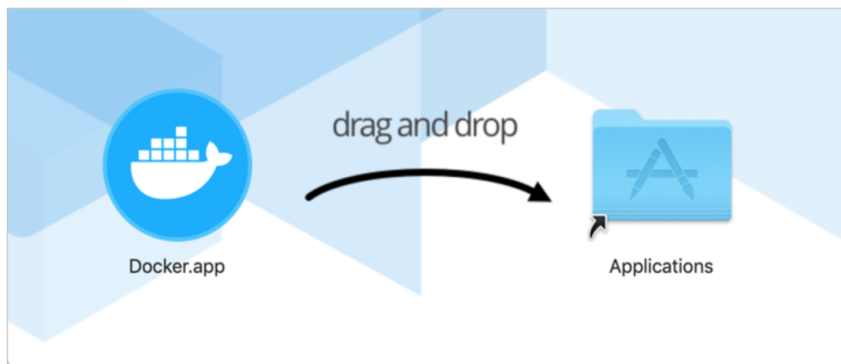
1-2. OS X へのインストール

(2) ダウンロードした Docker.dmg を開き、Docker.app を Applicationsフォルダにドラッグする。

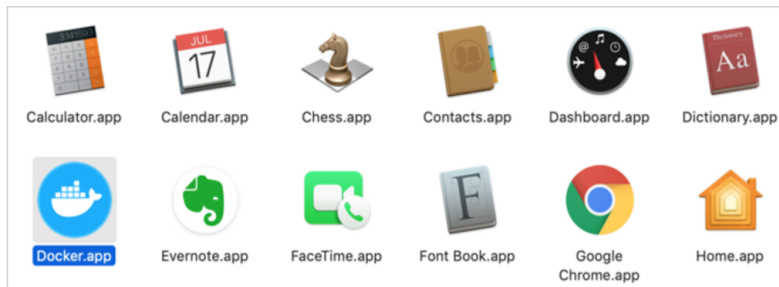
Docker Desktop の Mac へのインストールと実行

対話的なインストール

1. Docker.dmg をダブルクリックしてインストーラーを開きます。Docker アイコンをアプリケーションフォルダーへドラッグします。



2. アプリケーションフォルダー内の Docker.app をダブルクリックして Docker を起動します。以下の例において、アプリケーションフォルダーは「グリッド」表示モードにしています。



Docker 演習

2. dockerコマンドによるイメージ取得とコンテナ稼働

(1) docker search [イメージ名]で、docker イメージを探す。

```
docker search ubuntu
```

```
PS C:\Users\haru1> docker search ubuntu
NAME                DESCRIPTION                STARS    OFFICIAL    AUTOMATED
ubuntu              Ubuntu is a Debian-based Linux operating sys... 16641    [OK]
websphere-liberty   WebSphere Liberty multi-architecture images ... 297      [OK]
open-liberty        Open Liberty multi-architecture images based... 62       [OK]
neurodebian         NeuroDebian provides neuroscience research s... 105      [OK]
ubuntu-debootstrap  DEPRECATED; use "ubuntu" instead                52       [OK]
ubuntu-upstart      DEPRECATED, as is Upstart (find other proces... 115      [OK]
ubuntu/nginx        Nginx, a high-performance reverse proxy & we... 103
ubuntu/squid         Squid is a caching proxy for the Web. Long-t... 71
ubuntu/cortex        Cortex provides storage for Prometheus. Long... 4
ubuntu/apache2       Apache, a secure & extensible open-source HT... 65
ubuntu/prometheus    Prometheus is a systems and service monitori... 53
ubuntu/kafka         Apache Kafka, a distributed event streaming ... 37
ubuntu/bind9         BIND 9 is a very flexible, full-featured DNS... 65
ubuntu/mysql         MySQL open source fast, stable, multi-thread... 54
ubuntu/zookeeper     ZooKeeper maintains configuration informatio... 12
ubuntu/postgres      PostgreSQL is an open source object-relation... 31
ubuntu/redis         Redis, an open source key-value store. Long-... 21
ubuntu/jre           Distroless Java runtime based on Ubuntu. Lon... 9
ubuntu/dotnet-aspnet Chiselled Ubuntu runtime image for ASP.NET a... 13
ubuntu/grafana        Grafana, a feature rich metrics dashboard & ... 9
ubuntu/memcached      Memcached, in-memory keyvalue store for smal... 5
ubuntu/dotnet-deps    Chiselled Ubuntu for self-contained .NET & A... 11
ubuntu/prometheus-alertmanager Alertmanager handles client alerts from Prom... 9
ubuntu/dotnet-runtime Chiselled Ubuntu runtime image for .NET apps... 13
ubuntu/cassandra      Cassandra, an open source NoSQL distributed ... 2
```

Docker 演習

2. dockerコマンドによるイメージ取得とコンテナ稼働

(2) docker pull [イメージ名]で、docker イメージを取得する。

```
docker pull ubuntu
```

```
PS C:\Users\haru1> docker pull ubuntu
Using default tag: latest
latest: Pulling from library/ubuntu
Digest: sha256:2b7412e6465c3c7fc5bb21d3e6f1917c167358449fecac8176c6e496e5c1f05f
Status: Image is up to date for ubuntu:latest
docker.io/library/ubuntu:latest
```

Docker 演習

2. dockerコマンドによるイメージ取得とコンテナ稼働

(3) コンテナを起動する。

```
docker run -it ubuntu /bin/bash
```

```
What's Next?
```

```
View a summary of image vulnerabilities and recommendations → docker scout quickview ubuntu
```

```
PS C:\Users\haru1> docker run -it ubuntu /bin/bash
```

```
root@63e1e453f424:/# |
```

Docker 演習

2. dockerコマンドによるイメージ取得とコンテナ稼働

(4) 稼働中のコンテナを確認する。

```
docker ps
```

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

新機能と改善のために最新の PowerShell をインストールしてください!https://aka.ms/PSWindows

PS C:\Users\haru1> docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS        NAMES
63e1e453f424   ubuntu   "/bin/bash"             19 seconds ago Up 18 seconds          nervous_kalam
PS C:\Users\haru1> |
```

※docker psコマンドはコンテナ内に存在しないため、コンテナ内部でこのコマンドを実行しようとするエラーが発生します。
コンテナ内で作業している場合は、まず exitコマンドでコンテナから抜け、ホストマシンのプロンプトに戻ってから
docker psコマンドを実行してください。または、新しいターミナルウィンドウを開いて、コマンドを実行することもできます。

Docker 演習

3. Dockerfileからのイメージの作成およびコンテナの起動・停止を行う。

(1) Dockerfileを作成し、必要な設定を記述する。

```
FROM python:3.7.5-slim
LABEL author="DeepSquare"
RUN pip install Flask requests
COPY ./server.py /server.py
ENV PORT 80
CMD ["python", "-u", "server.py"]
```

Dockerfile の作成の流れ

1. ベースイメージの指定
2. イメージ情報を登録
3. パッケージのインストール
4. ソースコードをコピー
5. デフォルト環境変数の指定
6. 実行コマンドの指定

(2) Dockerfileという名前のファイルに保存する。.dockerfileという拡張子をつける必要はない。

Docker 演習

3. Dockerfileからのイメージの作成およびコンテナの起動・停止を行う。

(3) Pythonのサーバーを作成してserver.pyという名前で保存する。

```
import os
from http.server import SimpleHTTPRequestHandler,
HTTPServer

PORT = int(os.environ.get('PORT', 80))

class SimpleHandler(SimpleHTTPRequestHandler):
    def do_GET(self):
        self.send_response(200)
        self.send_header('Content-type', 'text/html')
        self.end_headers()
        self.wfile.write(b"Hello, Docker!")

if __name__ == "__main__":
    with HTTPServer(("", PORT), SimpleHandler) as httpd:
        print(f"Serving on port {PORT}")
        httpd.serve_forever()
```

※server.pyとDockerfileは同じディレクトリに配置する必要があります。



Docker 演習

3. Dockerfileからのイメージの作成およびコンテナの起動・停止を行う。

(4) Docker イメージを作成する。

```
docker build -t mypythonserver:latest .
```

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

新機能と改善のために最新の PowerShell をインストールしてください!https://aka.ms/PSWindows

PS C:\Users\haru1> docker build -t mypythonserver:latest .
[+] Building 9.2s (9/9) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile            0.0s
=> => transferring dockerfile: 199B                             0.0s
=> [internal] load .dockerignore                               0.0s
=> => transferring context: 2B                                    0.0s
=> [internal] load metadata for docker.io/library/python:3.7.5-slim 3.4s
=> [auth] library/python:pull token for registry-1.docker.io    0.0s
=> CACHED [1/3] FROM docker.io/library/python:3.7.5-slim@sha256:59af1bb7fb92ff97c9a23abae23f6bed13a95dbfd8100c7 0.0s
=> [internal] load build context                                0.0s
=> => transferring context: 548B                                  0.0s
=> [2/3] RUN pip install Flask requests                        5.7s
=> [3/3] COPY ./server.py /server.py                          0.0s
=> exporting to image                                           0.1s
=> => exporting layers                                           0.1s
=> => writing image sha256:dcd422a80e7dd7131f521e5d8d1d820e839e21933d9a6d49cba268d8b5cd2dca 0.0s
=> => naming to docker.io/library/mypythonserver:latest        0.0s
```

Docker 演習

3. Dockerfileからのイメージの作成およびコンテナの起動・停止を行う。

(5) コンテナを起動する。

```
docker run -d -p 80:80 mpythonservice:latest
```

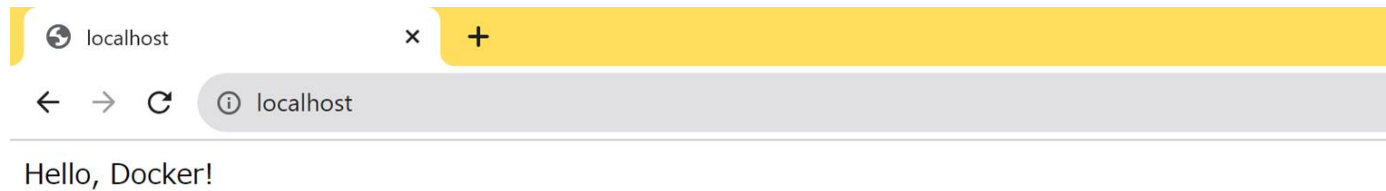
What's Next?

View a summary of image vulnerabilities and recommendations → `docker scout quickview`
PS C:\Users\haru1> `docker run -d -p 80:80 mpythonservice:latest`
`577e7807d46e589669d023ea046ecb9038f284c62e0d275f04c28c8bad1bc591`

Docker 演習

3. Dockerfileからのイメージの作成およびコンテナの起動・停止を行う。

(6) ブラウザで<http://localhost>にアクセスして、"Hello, Docker!"というメッセージが表示されることを確認する。



Docker 演習

3. Dockerfileからのイメージの作成およびコンテナの起動・停止を行う。

(7) コンテナを停止する。

稼働中のコンテナIDを確認する。

```
docker ps
```

上記コマンドで表示されたコンテナIDを使用して、コンテナを停止する。

```
docker stop [コンテナID]
```

```
PS C:\Users\haru1> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                    NAMES
577e7807d46e   mpythonserv:latest  "python -u server.py"   40 seconds ago  Up 39 seconds  0.0.0.0:80->80/tcp      dreamy_dirac
63e1e453f424   ubuntu        "/bin/bash"             44 minutes ago  Up 44 minutes  -                        nervous_kalam
PS C:\Users\haru1> docker stop 577e7807d46e
577e7807d46e
PS C:\Users\haru1> |
```

Docker 演習

4. 深層学習向けフレームワークが利用できるDocker環境上でモデルの訓練を行う。

4-1. PyTorchの場合

(1) 深層学習フレームワーク (PyTorch) を含むイメージを取得または作成する。

```
docker pull pytorch/pytorch:latest
```

```
PS C:\Users\haru1> docker pull pytorch/pytorch:latest
latest: Pulling from pytorch/pytorch
Digest: sha256:b32443a58a60c4ca4d10651e3c5b2aa7211ffcfbdc433593d71b53c2f2e94703
Status: Image is up to date for pytorch/pytorch:latest
docker.io/pytorch/pytorch:latest

What's Next?
  View a summary of image vulnerabilities and recommendations → docker scout quickview pytorch/pytorch:latest
PS C:\Users\haru1> |
```

Docker 演習

4. 深層学習向けフレームワークが利用できるDocker環境上でモデルの訓練を行う。

(2) ホスト側のディレクトリを環境変数に設定する。

```
$env:HOST_DIR = "C:/home/user/my_project" #windowsの場合  
export HOST_DIR = /home/user/my_project #mac、Linuxの場合
```

※C:/home/user/my_project は例として挙げられたディレクトリです。実際の環境に合わせて適切なパスを指定してください。

```
What's Next?  
View a summary of image vulnerabilities and recommendations → docker scout quickview pytorch/pytorch:latest  
PS C:\Users\haru1> $env:HOST_DIR = "C:/Users/haru1/my_project"  
PS C:\Users\haru1>
```



※学習を行うための jupyterソースファイルを用意しておきます。

例：



04_lecture_pytorch_cifar10_finetuning.ipynb

ダウンロード用リンク

<https://x.gd/UvBtm>

Docker 演習

4. 深層学習向けフレームワークが利用できるDocker環境上でモデルの訓練を行う。

(3) `docker run -it [オプション] [ホストとコンテナのポートマッピング] -v [ホスト側のデータセットパス]:[コンテナ内のマウント先パス] [イメージ名]:[タグ]`でコンテナを起動する。

```
docker run -it --rm -p 8888:8888 -v "${env:HOST_DIR}:/workspace" pytorch/pytorch:latest  
#windowsの場合
```

```
docker run -it --rm -p 8888:8888 -v "$HOST_DIR:/workspace" pytorch/pytorch:latest  
#mac、Linuxの場合
```

オプション : `--rm`

ホストとコンテナのポートマッピング : `-p 8888:8888`

ホスト側のデータセットパス : `/home/user/my_project`

コンテナ内のマウント先パス : `/workspace`

イメージ名:タグ : `pytorch/pytorch:latest`

What's Next?

```
View a summary of image vulnerabilities and recommendations → docker scout quickview pytorch/pytorch:latest  
PS C:\Users\haru1> $env:HOST_DIR = "C:/Users/haru1/my_project"  
PS C:\Users\haru1> docker run -it --rm -p 8888:8888 -v "${env:HOST_DIR}:/workspace" pytorch/pytorch:latest  
root@6e957ffc55e8:/workspace# |
```


Docker 演習

4. 深層学習向けフレームワークが利用できるDocker環境上でモデルの訓練を行う。

(4) Jupyter Notebook をインストールする。

```
pip install jupyter
```

```
root@01b9a1787dac:/workspace# pip install jupyter
Collecting jupyter
  Downloading jupyter-1.0.0-py2.py3-none-any.whl (2.7 kB)
Collecting notebook (from jupyter)
  Downloading notebook-7.0.6-py3-none-any.whl.metadata (10 kB)
Collecting qtconsole (from jupyter)
  Downloading qtconsole-5.5.1-py3-none-any.whl.metadata (5.1 kB)
Collecting jupyter-console (from jupyter)
  Downloading jupyter_console-6.6.3-py3-none-any.whl (24 kB)
Collecting nbconvert (from jupyter)
  Downloading nbconvert-7.12.0-py3-none-any.whl.metadata (7.7 kB)
Collecting ipykernel (from jupyter)
  Downloading ipykernel-6.27.1-py3-none-any.whl.metadata (6.3 kB)
Collecting ipywidgets (from jupyter)
  Downloading ipywidgets-8.1.1-py3-none-any.whl.metadata (2.4 kB)
Collecting comm>=0.1.1 (from ipykernel->jupyter)
```

Docker 演習

4. 深層学習向けフレームワークが利用できるDocker環境上でモデルの訓練を行う。

(5) Jupyter Notebook を起動する。

```
jupyter notebook --ip 0.0.0.0 --port 8888 --no-browser --allow-root
```

```
root@01b9a1787dac:/workspace# jupyter notebook --ip 0.0.0.0 --port 8888 --no-browser --allow-root
[I 2023-12-06 01:35:48.516 ServerApp] Package notebook took 0.0000s to import
[I 2023-12-06 01:35:48.520 ServerApp] Package jupyter_lsp took 0.0040s to import
[W 2023-12-06 01:35:48.520 ServerApp] A `_jupyter_server_extension_points` function was not found in jupyter_lsp. Instead, a `_jupyter_server_extension_paths` function was found and will be used for now. This function name will be deprecated in future releases of Jupyter Server.
[I 2023-12-06 01:35:48.523 ServerApp] Package jupyter_server_terminals took 0.0029s to import
[I 2023-12-06 01:35:48.523 ServerApp] Package jupyterlab took 0.0000s to import
[I 2023-12-06 01:35:48.540 ServerApp] Package notebook_shim took 0.0000s to import
[W 2023-12-06 01:35:48.540 ServerApp] A `_jupyter_server_extension_points` function was not found in notebook_shim. Instead, a `_jupyter_server_extension_paths` function was found and will be used for now. This function name will be deprecated in future releases of Jupyter Server.
[I 2023-12-06 01:35:48.540 ServerApp] jupyter_lsp | extension was successfully linked.
[I 2023-12-06 01:35:48.542 ServerApp] jupyter_server_terminals | extension was successfully linked.
[I 2023-12-06 01:35:48.545 ServerApp] jupyterlab | extension was successfully linked.
[I 2023-12-06 01:35:48.547 ServerApp] notebook | extension was successfully linked.
[I 2023-12-06 01:35:48.547 ServerApp] Writing Jupyter server cookie secret to /root/.local/share/jupyter/runtime/jupyter_cookie_secret
[I 2023-12-06 01:35:48.730 ServerApp] notebook_shim | extension was successfully linked.
[I 2023-12-06 01:35:48.738 ServerApp] notebook_shim | extension was successfully loaded.
[I 2023-12-06 01:35:48.740 ServerApp] jupyter_lsp | extension was successfully loaded.
[I 2023-12-06 01:35:48.741 ServerApp] jupyter_server_terminals | extension was successfully loaded.
[I 2023-12-06 01:35:48.742 LabApp] JupyterLab extension loaded from /opt/conda/lib/python3.10/site-packages/jupyterlab
[I 2023-12-06 01:35:48.742 LabApp] JupyterLab application directory is /opt/conda/share/jupyter/lab
[I 2023-12-06 01:35:48.742 LabApp] Extension Manager is 'pypi'.
[I 2023-12-06 01:35:48.744 ServerApp] jupyterlab | extension was successfully loaded.
```

Docker 演習

4. 深層学習向けフレームワークが利用できるDocker環境上でモデルの訓練を行う。

(6) Jupyter Notebook の起動後、表示されるURLをクリックして、Jupyter Notebook にアクセスする。

```
[I 2023-12-06 01:35:48.738 ServerApp] notebook_shim | extension was successfully linked.
[I 2023-12-06 01:35:48.738 ServerApp] notebook_shim | extension was successfully loaded.
[I 2023-12-06 01:35:48.740 ServerApp] jupyter_lsp | extension was successfully loaded.
[I 2023-12-06 01:35:48.741 ServerApp] jupyter_server_terminals | extension was successfully loaded.
[I 2023-12-06 01:35:48.742 LabApp] JupyterLab extension loaded from /opt/conda/lib/python3.10/site-packages/jupyterlab
[I 2023-12-06 01:35:48.742 LabApp] JupyterLab application directory is /opt/conda/share/jupyter/lab
[I 2023-12-06 01:35:48.742 LabApp] Extension Manager is 'pypi'.
[I 2023-12-06 01:35:48.744 ServerApp] jupyterlab | extension was successfully loaded.
[I 2023-12-06 01:35:48.746 ServerApp] notebook | extension was successfully loaded.
[I 2023-12-06 01:35:48.746 ServerApp] Serving notebooks from local directory: /workspace
[I 2023-12-06 01:35:48.746 ServerApp] Jupyter Server 2.12.0 is running at:
[I 2023-12-06 01:35:48.746 ServerApp] http://01b9a1787dac:8888/tree?token=96d61be407fd5a3f9aba91eab0fc7e8de2354534ed2e5a
cb
[I 2023-12-06 01:35:48.746 ServerApp] → http://127.0.0.1:8888/tree?token=96d61be407fd5a3f9aba91eab0fc7e8de2354534ed2e5
acb
[I 2023-12-06 01:35:48.746 ServerApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirm
ation).
[C 2023-12-06 01:35:48.747 ServerApp]

To access the server, open this file in a browser:
file:///root/.local/share/jupyter/runtime/jpserver-40-open.html
Or copy and paste one of these URLs:
http://01b9a1787dac:8888/tree?token=96d61be407fd5a3f9aba91eab0fc7e8de2354534ed2e5a
cb
→ http://127.0.0.1:8888/tree?token=96d61be407fd5a3f9aba91eab0fc7e8de2354534ed2e5a
cb
[I 2023-12-06 01:35:48.756 ServerApp] Skipped non-installed server(s): bash-language-server, dockerfile-language-server-
nodejs, javascript-typescript-langserver, jedi-language-server, julia-language-server, pyright, python-language-server,
python-lsp-server, r-language-server, sql-language-server, texlab, typescript-language-server, unified-language-server, v
scode-css-languageserver-bin, vscode-html-languageserver-bin, vscode-json-languageserver-bin, yaml-language-server
```

 jupyter

File View Settings Help

Files

Running

Select items to perform actions on them.

New Upload

/

Name	Last Modified	File Size
data	1 hour ago	
04_lecture_pytorch_cifar10_finetuning.ipynb	1 hour ago	605.1 KB
cifar_net.pth	1 hour ago	512.3 MB
desktop.ini	6 hours ago	114 B

 jupyter 04_lecture_pytorch_cifar10_finetuning Last Checkpoint: 18 days ago

File Edit View Run Kernel Settings Help

Not Trusted

JupyterLab Python 3 (ipykernel)

```
# ネットワークを通じてイメージを渡し、出力を得ます。
outputs = net(images)

# 出力から最大スコアを持つクラスを取得します (予測)。
_, predicted = torch.max(outputs, 1)

# 正解の予測を見つける
c = (predicted == labels).squeeze()

# バッチ内の各サンプルに対してループし、クラスごとの正解数と合計数を更新します。
# ここでは、バッチの最初の4つのサンプルのみを考慮しています。
for i in range(4):
    label = labels[i]
    class_correct[label] += c[i].item() # 正しい予測を更新
    class_total[label] += 1 # クラスごとの合計を更新

# 各クラスの正確さを計算し、表示します。
for i in range(10):
    print('Accuracy of %5s : %2d %%' % (
        classes[i], 100 * class_correct[i] / class_total[i]))
```

Accuracy of plane : 86 %
Accuracy of car : 91 %
Accuracy of bird : 83 %
Accuracy of cat : 71 %
Accuracy of deer : 81 %
Accuracy of dog : 75 %
Accuracy of frog : 86 %
Accuracy of horse : 79 %
Accuracy of ship : 94 %
Accuracy of truck : 89 %

Docker 演習

4. 深層学習向けフレームワークが利用できるDocker環境上でモデルの訓練を行う。

4-2. TensorFlowの場合

(1) 深層学習フレームワーク (TensorFlow) を含むイメージを取得または作成する。

```
docker pull tensorflow/tensorflow:latest
```

(2) データセットと学習スクリプトが含まれるディレクトリを指定する。

```
$env:HOST_DIR = "C:/home/user/my_project" #windowsの場合
```

```
export HOST_DIR = /home/user/my_project #macの場合
```

学習スクリプト : HOST_DIR=/home/user/my_project/train_script.py

データセット : HOST_DIR=/home/user/my_project/dataset/

※C:/home/user/my_project は例として挙げられたディレクトリです。実際の環境に合わせて適切なパスを指定してください。また、train_script.py の中身や、データセットの形式・内容に応じて、必要な Pythonライブラリや依存関係がある場合は、Dockerfile を作成してイメージをカスタマイズすることが必要です。

Docker 演習

4. 深層学習向けフレームワークが利用できるDocker環境上でモデルの訓練を行う。

(3) `docker run -it -v [ホスト側のデータセットパス]:[コンテナ内のマウント先パス] [イメージ名]:[タグ]`でコンテナを起動する。

```
docker run -it -v ${env:HOST_DIR}:/tf/my_project tensorflow/tensorflow:latest /bin/bash  
#windowsの場合
```

```
docker run -it -v $HOST_DIR:/tf/my_project tensorflow/tensorflow:latest /bin/bash  
#macの場合
```

ホスト側のデータセットパス : `/home/user/my_project`

コンテナ内のマウント先パス : `/tf/my_project`

イメージ名:タグ : `tensorflow/tensorflow:latest`

(4) コンテナ内で学習スクリプトを実行し、モデルの訓練を行う。

```
python /tf/my_project/train_script.py
```